Taller 7: pipeline para detectar y corregir errores médicos en notas clínicas (MEDIQA-CORR 2024)

Grupo 6
David Lasso Nestor Fadiño Camilo García
Procesamiento de Lenguaje Natural — Maestría
Código: `https://github.com/CamiloGarcia06/maestria-pln/tree/taller-7`

Noviembre de 2026

Resumen

Una nota clínica de MEDIQA-CORR 2024 es correcta o trae un solo error médico. Hay que decir si tiene error (subtarea A), en qué oración está (B) y cómo se corrige (C). Revisamos los artículos de MEDEC y de WangLab y el dataset de entrenamiento MS (2.189 notas). Proponemos un pipeline con dos modelos de Hugging Face: BiomedBERT detecta y localiza el error, oración por oración, y BioBART-v2 escribe la oración corregida. Cada hallazgo del dataset termina en una decisión del pipeline. Por ejemplo, las notas vienen en pares (versión correcta y versión con error del mismo caso), así que la partición se hace por caso. Para cada caja del pipeline damos las funciones a programar y las bases que el sistema tiene que superar.

1. Tarea y lo que tomamos de los artículos

Subtarea	Salida	Métrica oficial
A. Bandera de error	1 si la nota tiene error, 0 si no	exactitud
B. Oración con error	id de la oración, -1 si no hay error	exactitud
C. Corrección	oración corregida, `NA` si no hay error	ROUGE-1, BERTScore, BLEURT y su promedio

MEDEC (Ben Abacha et al., 2024) es el benchmark. Cada nota es correcta o trae un error de cinco tipos: diagnóstico, manejo, tratamiento, farmacoterapia u organismo causal. La parte MS, que es nuestro CSV, sale de preguntas del examen médico de EE. UU. (MedQA). Los anotadores pusieron en el caso la respuesta correcta o una incorrecta, casi siempre en la mitad o al final del texto. Con un *prompt*, Claude 3.5 Sonnet fue el mejor detectando (70 % en A) y o1-preview el mejor corrigiendo (0,70 de puntaje compuesto). De este artículo tomamos dos ideas. Primera: **detectar y corregir son habilidades distintas**, porque el mejor detector quedó último corrigiendo. Por eso usamos un modelo para cada tarea. Segunda: **ROUGE no ve sinónimos** («agammaglobulinemia de Bruton» y «ligada al X» son la misma enfermedad), así que medimos también con BERTScore y BLEURT.

WangLab (Toma et al., 2024) ganó las tres subtareas (86,5 % en A, 83,6 % en B, 0,789 en C) con GPT-4 y DSPy. Para notas reales encadena tres módulos (detectar → localizar → corregir) y aplica un **control de calidad**: si el ROUGE-L entre la oración original y la corregida baja de 0,7, descarta la corrección. Para MS, además, recupera con TF-IDF la pregunta de MedQA de la que salió la nota y convierte la corrección en elegir una de sus opciones. Tomamos la cadena de tres pasos y el control de calidad, pero con el umbral recalibrado (Sección 4). La recuperación de MedQA queda como extensión.

2. Revisión del dataset

`MEDIQA-CORR-2024-MS-TrainingData.csv` tiene 2.189 notas. Para cada nota trae el texto, las oraciones numeradas, la bandera, el id y el texto de la oración con error, y la oración corregida. No trae el tipo de error. Las cifras salen de `scripts/revisar_dataset.py`. Cada hallazgo de la Tabla 1 termina en una decisión del pipeline.

Tabla 1: Hallazgos del dataset y la decisión que toman en el pipeline.

Hallazgo	Cifra	Decisión
Clases casi balanceadas	1.219 con error (55,7 %) y 970 sin error	Decir siempre «hay error» da 55,7 % en A: es la base a superar
Muchas oraciones y una sola mala	12,6 oraciones por nota (máx. 48); 27.673 en total	El detector clasifica **cada oración**: 21,7 negativas por positiva *a nivel de oración* (por nota es 1.219 contra 970), así que se entrena con peso de clase $\approx 21{,}7$
Las notas vienen en pares	1.219 casos; en 967 hay una versión correcta y otra con error (88 % de las notas), en 250 solo la versión con error, en 1 solo la correcta y 1 grupo junta 4 notas de dos casos casi iguales	Partición **por caso** (`GroupKFold`). Si una versión cae en entrenamiento y la otra en validación, el modelo ya vio el caso y la validación miente. La primera oración no sirve de clave: junta casos distintos que abren igual (6 grupos de 3–4 notas) y separa 28 pares, 15 de ellos con el error en la oración 0
El error suele ir al final	62,3 % en la última oración; 9,4 % en la primera mitad	«Elegir siempre la última oración» es la base de B. No damos la posición como rasgo: es un efecto de cómo se armó MS y no se cumple en notas reales
Segmentación imperfecta	1.289 notas con «oraciones» de ≤ 3 palabras (`"Hg."`); 336 con 1.131 líneas sin id, casi todas de laboratorio	**No descartar nada**: las líneas sin id se pegan a la oración anterior. Unir los fragmentos, pero **guardar el mapa a los ids oficiales**, porque B se califica con ellos
La corrección es un cambio corto	3,5 palabras cambiadas en promedio (máx. 24)	El corrector reescribe **una oración**, no la nota. El control de calidad rechaza las salidas que se alejan demasiado de la original

Vocabulario contra largo de secuencia. Son dos cosas distintas. El *tamaño del vocabulario* es cuántos tokens conoce el tokenizador. Fija la tabla de *embeddings* (en BiomedBERT, $30.522 \times 768 = 23{,}4\,\text{M}$ de parámetros) y decide cómo se parte cada palabra. El *largo de secuencia* es cuántos tokens tiene un ejemplo. Fija el relleno y el corte (`max_length`), no los parámetros. La Tabla 2 compara los tokenizadores con las notas del dataset. El modelo general despedaza los términos médicos y algunas notas pasan de 512 tokens. Los dos biomédicos dejan las palabras casi enteras y ninguna nota se corta. Usamos `max_length=512` con relleno dinámico: cada lote se rellena hasta su nota más larga.

Tabla 2: Tokenizadores sobre las notas de MS. Ejemplo: «Haemophilus influenzae metoprolol».

Tokenizador	Vocab.	Partición del ejemplo	Tokens por nota (media / máx.)	Par oración + nota (máx.)	Largo máx. del modelo
BiomedBERT	30.522	`haem ##ophilus influenzae met ##oprol ##ol`	173 / 414	425	512
BioBART-v2	85.401	`Haemophilus influenzae metoprolol`	189 / 482	497	1.024
Flan-T5 (general)	32.100	12 piezas: `Ha e m ophil us …`	204 / 516	521	512

3. El pipeline

Usamos dos modelos de Hugging Face, uno para cada habilidad:

- **Detector-localizador (A y B):** BiomedBERT `microsoft/BiomedNLP-BiomedBERT-base-uncased-abstract-fulltext`, un BERT preentrenado desde cero con PubMed. Lo ajustamos para decir si *una oración*, leída junto a *toda la nota*, es la del error. La oración tiene que ir con la nota porque el error solo se ve en contexto. «El organismo causal es *H. influenzae*» es una frase correcta por sí sola. Es un error porque el cuadro de la nota (consolidación lobar, esputo verdoso) apunta a neumococo.
- **Corrector (C):** BioBART-v2 `GanjinZero/biobart-v2-large`, un BART codificador-decodificador preentrenado con PubMed. Recibe la nota con la oración sospechosa marcada y genera la oración corregida.

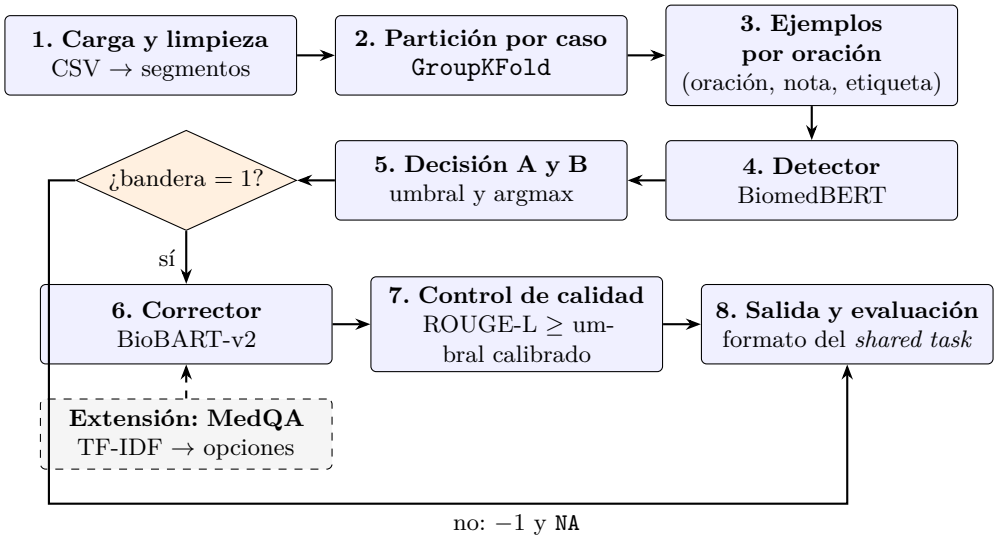


Figura 1: El pipeline. Las cajas 1–5 resuelven A y B; la 6 y la 7 solo corren si la nota tiene error.

La Tabla 3 da las funciones a programar en cada caja.

Tabla 3: Funciones a programar en cada caja del pipeline.

Función	Qué hace
Caja 1 · Carga y limpieza	
`cargar_corpus(ruta)`	Lee el CSV y normaliza tipos (id a entero, `NA` a `None`)
`parsear_oraciones(columna)`	Convierte `"0 texto\n1 texto"` en {id: oración}. Las líneas sin id (laboratorios) se pegan a la oración anterior, no se descartan
`unir_fragmentos(oraciones)`	Pega a la oración anterior los fragmentos que no son oración (`"Hg."`, `"100/68 mm"`)
`mapear_ids_oficiales(segmentos)`	Guarda qué ids oficiales cubre cada segmento, para devolver el id que exige B
Caja 2 · Partición por caso	
`identificar_casos(textos, oraciones)`	Une dos notas si comparten una oración y su texto coincide en $\geq 0{,}7$ (`SequenceMatcher`). La similitud tiene un hueco entre 0,4 y 0,8: el umbral no es delicado
`dividir_por_caso(df, k=5)`	`GroupKFold`: las dos versiones de un caso caen siempre del mismo lado. Las 574 notas de validación oficial de MS quedan como prueba final
Caja 3 · Ejemplos por oración	
`armar_pares_oracion_nota(nota)`	Por cada segmento: (oración candidata, nota completa, etiqueta 1 si contiene el id de error)

Función	Qué hace
`calcular_peso_clase(pares)`	negativos / positivos ≈ 21,7, para el `pos_weight` de la pérdida
Caja 4 · Detector BiomedBERT	
`tokenizar_pares(pares, tok)`	`[CLS] oración [SEP] nota [SEP]`, `truncation="only_second"`, `max_length=512`
`entrenar_detector(train, val)`	`AutoModelForSequenceClassification(num_labels=1)`: el vector `[CLS]` (768) pasa a 1 logit. `BCEWithLogitsLoss(pos_weight)`, AdamW 2e-5, lote 16, 3–4 épocas, *early stopping* con el F1 de A por nota
`puntuar_oraciones(modelo, nota)`	Probabilidad de error de cada oración de la nota
Caja 5 · Decisión de A y B	
`ajustar_umbral(probs, etiquetas)`	Barre umbrales en validación y se queda con el de mayor F1 de la bandera, calculado *por nota*: la probabilidad de la nota es la de su oración más sospechosa. Controla los falsos positivos que MEDEC vio en los LLM
`decidir_bandera(probs, umbral)`	1 si la oración más sospechosa supera el umbral
`elegir_oracion(probs, mapa_ids)`	Id oficial de la oración más sospechosa, o -1 si la bandera es 0
Caja 6 · Corrector BioBART-v2	
`marcar_oracion(nota, id)`	Encierra la oración entre `<err>` y `</err>` (tokens nuevos, `resize_token_embeddings`)
`armar_ejemplos_correccion(df)`	Entrada: la nota marcada. Objetivo: la oración corregida. Solo las 1.219 notas con error
`entrenar_corrector(train, val)`	`AutoModelForSeq2SeqLM` con `Seq2SeqTrainer`, tasa 3e-5, lote 8, hasta 5 épocas, mejor punto según ROUGE-1 en validación
`generar_correccion(modelo, nota)`	*Beam search* con 4 haces y `max_new_tokens=96`
Caja 7 · Control de calidad	
`controlar_correccion(orig, haces, umbral)`	Si el ROUGE-L con la original baja del umbral (reescribió de más), prueba el siguiente de los 4 haces; si ninguno pasa, queda el primero. Si la salida es igual a la original, la marca para revisión
`calibrar_umbral_calidad(val)`	Elige el umbral con mayor puntaje de C en validación. El 0,7 de WangLab rechazaría el 57 % de las correcciones correctas
Caja 8 · Salida y evaluación	
`formatear_salida(id, bandera, …)`	Línea `text_id bandera id corrección`, con `-1 NA` si no hay error
`exactitud_bandera(pred, real)`	Subtareas A y B (métrica oficial). En B, acertar el -1 también
`exactitud_oracion(pred, real)`	cuenta; aparte, B solo sobre las notas con error
`metricas_bandera(pred, real)`	Precisión, *recall*, F1 y matriz de confusión de A: separa falsos positivos de falsos negativos
`puntajes_correccion(pred, ref)`	ROUGE-1, BERTScore (`deberta-xlarge-mnli`), BLEURT-20 y su promedio
`puntaje_compuesto(pred, ref)`	Regla oficial: 1 si ambas son `NA`, 0 si solo una lo es; si no, la métrica
Extensión · MedQA	
`indexar_medqa(dataset)`	Índice TF-IDF de `GBaker/MedQA-USMLE-4-options`
`recuperar_pregunta(nota, k=3)`	Las preguntas de MedQA más parecidas a la nota
`candidatos_correccion(pregunta)`	Sus opciones de respuesta: pistas para el corrector o para reordenar sus haces

Tabla 4: Hiperparámetros y de dónde sale cada valor. El vocabulario fija parámetros; el largo de secuencia, relleno y corte.

Hiperparámetro	Detector BiomedBERT	Corrector BioBART-v2	De dónde sale
Dimensión, capas, cabezas	768, 12, 12×64	1.024, 12+12, 16×64	`config.json`; fijos
Vocabulario	30.522	$85.401 + 2 = 85.403$	Fija los *embeddings*
Largo de secuencia	512 (par máx. 425)	1.024 (máx. $\approx$ 484); salida 96	Medido: nada se corta
Lote	16	8	Memoria (modelo *large*)
Optimizador, tasa	AdamW, 2e-5	AdamW, 3e-5	Típico de ajuste fino
Épocas	3–4, parada por F1 de A	Hasta 5, mejor ROUGE-1	Pocos datos: sobreajusta
Pérdida	BCE, `pos_weight` = 21,7	Entropía cruzada	26.454 / 1.219 oraciones
Decodificación	—	*Beam search*, 4 haces	*Greedy* es 1 haz
Umbral	Bandera: mayor F1	Calidad: mayor puntaje C	Calibrados en validación

4. Evaluación y limitaciones

Cada etapa se compara con una base sencilla y con lo publicado (Tabla 5). En B, «la última oración» ya acierta el 62 % de las notas con error. Si el detector no supera esa cifra, no mejora esa regla sencilla. La exactitud oficial no distingue falsas alarmas de errores perdidos: reportamos también precisión, *recall* y F1 de A, y B solo sobre las notas con error.

Tabla 5: Bases y resultados publicados. Las dos primeras filas se calcularon sobre nuestro entrenamiento; las demás, sobre la prueba de cada artículo.

Sistema	A. Bandera	B. Oración	C. Compuesto
Siempre «hay error»	55,7 %	—	—
Siempre «la última oración» (notas con error)	—	62,3 %	—
Claude 3.5 Sonnet, *prompt* (MEDEC)	70,2 %	65,6 %	0,280
o1-preview, *prompt* (MEDEC)	67,5 %	61,4 %	0,698
Médicos #1 / #2 (MEDEC)	79,6 / 71,6 %	65,9 / 66,8 %	0,453 / 0,712
WangLab, GPT-4 + DSPy	86,5 %	83,6 %	0,789

Limitaciones. Solo tenemos datos MS: las notas reales (UW) piden un acuerdo de uso. Un modelo ajustado con MS puede aprender la plantilla del examen y no los errores reales; en MEDEC, o1-preview baja de 73 % en MS a 58 % en UW. El CSV no trae el tipo de error, así que no podemos medir el *recall* por tipo. Las métricas de C castigan los sinónimos: hay que revisar a mano las correcciones con puntaje bajo. Y el control de calidad no se puede copiar: el ROUGE-L entre la oración con error y su corrección de referencia tiene mediana 0,667, así que el 0,7 de WangLab rechazaría 700 de las 1.219 correcciones *correctas* (57 %); «Proteus mirabilis» $\rightarrow$ «Klebsiella pneumoniae» da 0,667. Por eso el umbral se calibra en validación. Ese control mide cuánto cambió la oración, no si quedó bien. En una nota nueva no hay referencia: el sistema solo puede marcar las correcciones dudosas para revisión médica. BERTScore contra la referencia sirve únicamente al evaluar con datos etiquetados.

Referencias

Ben Abacha, A. et al. (2024). *MEDEC: A Benchmark for Medical Error Detection and Correction in Clinical Notes*. arXiv:2412.19260. Datos: `https://github.com/abachaa/MEDEC`.

Toma, A. et al. (2024). *WangLab at MEDIQA-CORR 2024: Optimized LLM-based Programs for Medical Error Detection and Correction*. Proceedings of the 6th Clinical Natural Language Processing Workshop, 616–623.